

Arbol de decision implementado sin framework

Clasificacion de pasajeros del dataset Spaceship Titanic

Entregable:	Implementacion de una tecnica de aprendizaje maquina sin el uso de un framework
Materia:	Inteligencia artificial avanzada para la ciencia de datos
Alumno:	Yael Michel García López
Matricula:	A01750911
Fecha:	02/09/2026
Repositorio:	https://github.com/yaelgl/Momento_Retroalimentacion_A01750911.git

Resultado principal

El arbol alcanza una exactitud de 77.48% sobre 2607 pasajeros que nunca vio durante el entrenamiento, contra 50.36% de un clasificador que siempre predice la clase mas frecuente. La implementacion pasa las 72 pruebas de correctitud incluidas en la entrega.

1. Objetivo

Programar manualmente un algoritmo de aprendizaje maquina, sin bibliotecas del ramo, entrenarlo con un conjunto de datos y comprobar que aprende y puede predecir. El algoritmo elegido es un arbol de decision para clasificacion, siguiendo la idea de CART. Se eligio uno supervisado y no un metodo de agrupamiento porque el entregable pide justificar los resultados con una matriz de confusion, y eso requiere una respuesta correcta contra la cual comparar cada prediccion.

La implementacion usa unicamente la libreria estandar de Python: csv, math, random y collections. No se importa scikit-learn, pandas ni numpy en ninguna parte del codigo que aprende o predice. El arbol se construye en 1.137 segundos.

2. Datos de entrenamiento y de prueba

Se usa el dataset Spaceship Titanic de Kaggle. Cada fila es un pasajero y la variable a predecir es Transported: si fue transportado a otra dimension o no. Es clasificacion binaria.

Kaggle reparte train.csv y test.csv, pero el segundo no sirve para evaluar porque no incluye la columna Transported: las respuestas se quedan en el servidor de la competencia. Por eso la evaluacion se hace partiendo train.csv, el unico archivo con etiquetas. De sus 8693 pasajeros se aparto el 30% como conjunto de prueba, con particion estratificada y semilla 42 para que sea reproducible.

Conjunto	Pasajeros	Transportados	Proporcion	Uso
Entrenamiento	6086	3065	50.36%	Construir el arbol
Prueba	2607	1313	50.36%	Medir el desempenio
Total (train.csv)	8693	4378	50.36%	-

La proporcion de transportados es identica en ambos conjuntos, y las clases estan balanceadas casi al 50%. Ese dato importa despues para justificar las metricas. Un detalle de procedimiento: primero se divide y despues se imputan los faltantes, para que las medianas y modas del relleno salgan solo del entrenamiento. Al revés habria fuga de informacion y las metricas saldrian infladas.

3. Preparacion de los datos

Se alimentan 15 variables: 8 numericas y 7 categoricas. Algunas son derivadas: Cabin viene como "B/O/P" y al partirla se obtienen Deck y Side; PassengerId como "0001_01", de donde salen TamGrupo y ViajaSolo; ademas se agregan GastoTotal y EsNino.

Los faltantes se rellenan segun la columna, empezando por la informacion mas confiable. El caso mas claro es CryoSleep: si el pasajero registro consumo es imposible que estuviera dormido, asi que el valor se deduce en lugar de estimarse. Para HomePlanet se aprovecha que el planeta nunca varia dentro de un grupo de viaje. Solo cuando no hay mejor informacion se usa la moda o la mediana.

No hizo falta escalar las variables ni convertir las categoricas a columnas binarias. Un arbol parte con preguntas como "Age <= 27", y esa comparacion no cambia si la columna se multiplica por una constante. Es una ventaja real de los arboles frente a los modelos lineales.

4. Como funciona el algoritmo

Cada nodo interno es una pregunta con dos respuestas y cada hoja es una prediccion. Para clasificar se empieza en la raiz y se baja respondiendo hasta llegar a una hoja. Lo que hay que decidir en cada nodo es cual es la mejor pregunta, y para eso se mide que tan mezcladas estan las clases con el indice de Gini:

$$\text{Gini} = 1 - \sum (p_k^2)$$

p_k es la proporcion de la clase k en el nodo. Vale 0 si todos los ejemplos son de la misma clase (nodo puro) y 0.5 si estan mitad y mitad, que es lo peor con dos clases. La implementacion incluye tambien la entropia de Shannon como alternativa: mide lo mismo y da arboles casi iguales.

Se prueban todas las divisiones posibles. Para una variable numerica, umbrales en el punto medio entre valores consecutivos distintos; para una categorica, preguntas del tipo "es igual a esta categoria". A cada una se le calcula la ganancia:

$$\text{ganancia} = \text{Gini}(\text{padre}) - [(n_{\text{izq}}/n) * \text{Gini}(\text{izq}) + (n_{\text{der}}/n) * \text{Gini}(\text{der})]$$

Se elige la de mayor ganancia y el proceso se repite en cada rama. Para que la busqueda no sea lenta, los ejemplos se ordenan una sola vez por variable y se recorren moviendo ejemplos de una rama a otra, en lugar de recomtar las clases para cada umbral.

Sin limites el arbol sigue partiendo hasta aislar cada ejemplo y termina memorizando en lugar de aprender. Los frenos implementados son cuatro: profundidad maxima, minimo de ejemplos para dividir, minimo por hoja y ganancia minima. Un nodo ya puro nunca se divide.

5. Eleccion de los hiperparametros

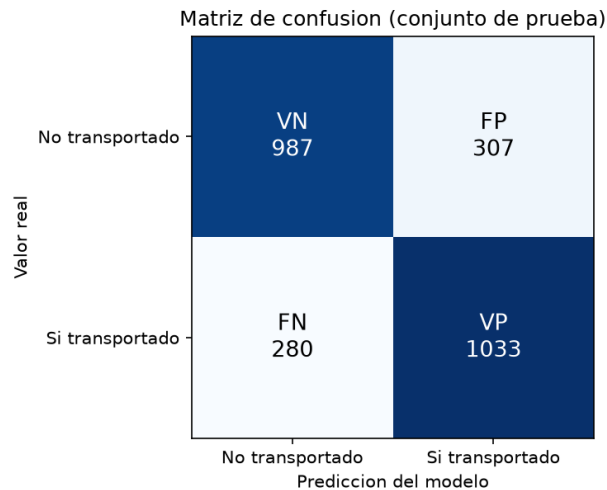
Hay una trampa facil de cometer: probar varias profundidades y quedarse con la que da mejor exactitud en prueba. Si se hace asi, el conjunto de prueba deja de ser una medida honesta. Para evitarlo, el conjunto de entrenamiento se vuelve a partir en sub-entrenamiento (80%) y validacion (20%), se prueban 60 combinaciones, se elige la mejor segun validacion y solo entonces se mide una vez en prueba.

Hiperparametro	Valor	Que controla
Profundidad maxima	9	Niveles de preguntas permitidos
Minimo para dividir	80	Tamano minimo de un nodo divisible
Minimo por hoja	40	Ejemplos que conserva cada hoja
Criterio	gini	Medida de impureza

Mirando el conjunto de prueba se llegaria a un valor algo mejor. Se reporta 77.48%, el resultado del procedimiento correcto, y no el numero mas favorable.

6. Resultados

La matriz cruza las 2607 predicciones del conjunto de prueba contra lo que realmente paso. La clase positiva es "si fue transportado".



Los cuatro conteos son: 987 verdaderos negativos, 1033 verdaderos positivos, 307 falsos positivos y 280 falsos negativos. O sea que el modelo acierta 2020 de 2607 casos, y los dos tipos de error estan parejos (307 contra 280), lo que indica que no esta inclinado hacia una clase.

Metrica	Formula	Prueba	Entrenamiento
Exactitud	$(VP+VN)/total$	77.48%	80.51%
Precision	$VP/(VP+FP)$	77.09%	81.02%
Sensibilidad	$VP/(VP+FN)$	78.67%	80.07%
Especificidad	$VN/(VN+FP)$	76.28%	80.97%
F1	media armonica	77.87%	80.54%

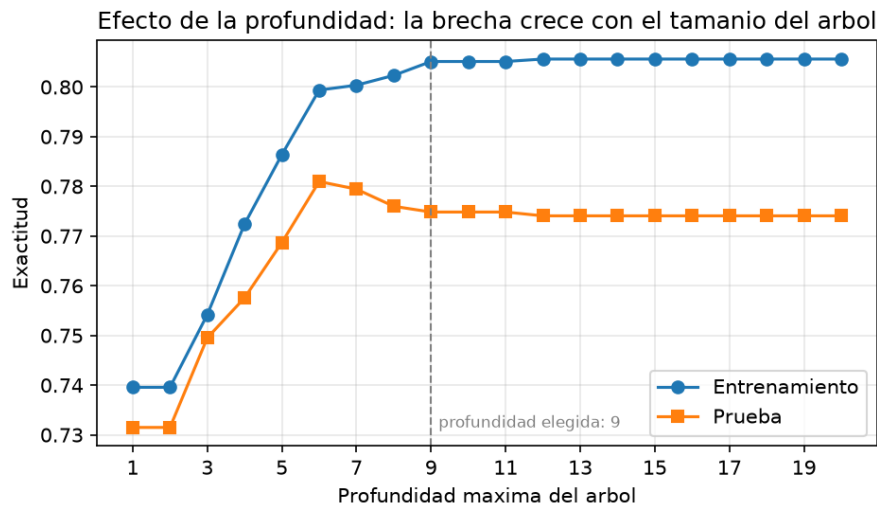
La exactitud es la metrica principal porque las clases estan balanceadas; en un dataset desbalanceado seria enganiosa, ya que con 95% de una clase bastaria predecir siempre esa para presumir 95% sin haber aprendido nada. Pero la exactitud sola no dice como se reparten los errores, y por eso se agregan las demas. Reportar sensibilidad (78.67%) y especificidad (76.28%) juntas muestra que el modelo funciona parecido en ambas clases, con 2.40 puntos de diferencia. El F1 resume precision y sensibilidad, y usa la media armonica porque castiga los desequilibrios.

Como referencia, un clasificador que siempre prediga la clase mas frecuente obtendria 50.36%, o sea el balance de las clases. El arbol logra 77.48%, 27.12 puntos por encima. Esa comparacion es la evidencia de que aprendio estructura real de los datos.

Sobreajuste

El arbol tiene 171 nodos, 86 de ellos hojas, y 9 niveles de profundidad. Acierta 80.51% en entrenamiento y 77.48% en prueba: una brecha de 3.03 puntos. Que sea positiva es normal, al modelo siempre le va mejor con los datos que ya conoce; lo importante es que sea chica.

La grafica muestra por que hay que limitar la profundidad: al dejar crecer el arbol la exactitud en entrenamiento sigue subiendo, pero en prueba se estanca. Esa separacion es el sobreajuste, y verla confirma que los criterios de paro sirven.



7. Comprobación de que la implementación es correcta

Una exactitud razonable no demuestra que el algoritmo esté bien programado: un algoritmo con errores también puede dar un número aceptable. Por eso el archivo `pruebas.py` incluye 72 verificaciones contra respuestas conocidas de antemano, calculadas a mano o derivadas de reglas con las que se generaron los datos a propósito. Todas pasan.

Que se comprueba	Como se verifica
Gini y entropía	Valores calculados a mano (50/50 da 0.5, puro da 0)
Matriz de confusión	Ejemplo de 10 casos contados uno por uno
Las seis métricas	Fraciones exactas de ese ejemplo (7/10, 3/5, 3/4)
Regla $y = 1$ si $x > 5$	Debe hallar el umbral 5.5 con una sola pregunta
Regla ($a > 0$ y $b > 0$)	Debe acertar los 121 casos
Relación tipo XOR	Exige combinar variables, no basta una
Regla categórica	Debe aislar justamente la categoría 'rojo'
Variables de ruido	La importancia debe quedarse en la que tiene señal
Categoría nunca vista	No debe provocar un error al predecir
Límites de crecimiento	Ninguna hoja menor al mínimo ni rama más honda
Determinismo	Entrenar dos veces da el mismo árbol
Reproducibilidad	La misma semilla da la misma partición

Las pruebas con reglas conocidas son las más informativas. Con datos generados a partir de "la clase es 1 cuando x es mayor que 5", el árbol no solo debe clasificar bien: debe hacerlo con una sola pregunta y poner el umbral en 5.5. Si el cálculo de la ganancia tuviera un error, elegiría otra variable u otro corte y la prueba lo detectaría.

Un error que las pruebas encontraron

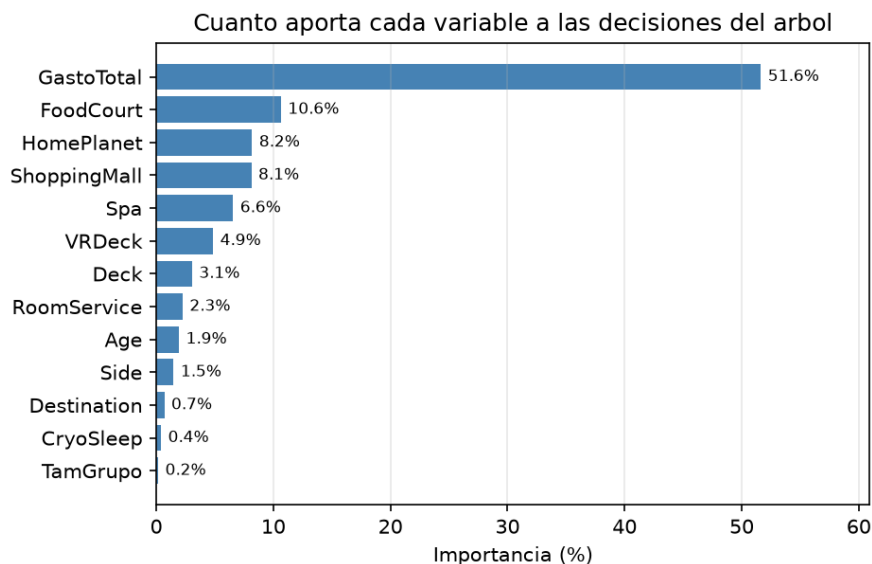
La prueba de XOR corrigió un defecto real. Ahí la clase es 1 cuando dos variables difieren, y ninguna por separado dice nada: partir por cualquiera deja las ramas mitad y mitad, o sea ganancia cero. La primera versión exigía ganancia estrictamente positiva, así que se detenía en la raíz y nunca aprendía la regla, aunque con dos niveles la resuelve. La corrección fue permitir divisiones de ganancia cero, igual que scikit-learn. El cambio no alteró ningún resultado del dataset real, solo agregó capacidad.

8. Que aprendio el modelo

Una ventaja de los arboles es que las reglas se pueden leer. Los tres primeros niveles del arbol entrenado:

```
GastoTotal <= 7.5 (n=6086, ganancia=0.1187)
|-- si: HomePlanet == Earth (n=2534, ganancia=0.0446)
    |-- si: Side == S (n=1240, ganancia=0.0058)
        |-- si: CryoSleep == False (n=639) [...]
        |-- no: Destination == TRAPPIST-1e (n=601) [...]
    |-- no: Deck == E (n=1294, ganancia=0.0295)
        |-- si: Age <= 31.5 (n=138) [...]
        |-- no: CryoSleep == False (n=1156) [...]
|-- no: FoodCourt <= 498 (n=3552, ganancia=0.0260)
    |-- si: ShoppingMall <= 402.5 (n=2560, ganancia=0.0394)
        |-- si: GastoTotal <= 1597 (n=1860) [...]
        |-- no: Spa <= 117.5 (n=700) [...]
    |-- no: Spa <= 1372.5 (n=992, ganancia=0.0501)
        |-- si: VRDeck <= 1173 (n=815) [...]
        |-- no: Spa <= 2927.5 (n=177) [...]
```

La primera pregunta es si el gasto total fue menor o igual a 7.5, o sea si el pasajero consumio algo. Tiene sentido: quienes viajaban en criosuenio no podian consumir, y son los que se transportaron con mas frecuencia. La grafica mide cuanta impureza elimina cada variable, pesada por cuantos pasajeros pasan por sus divisiones; GastoTotal concentra 51.64%.



Hay un resultado que sorprende: CryoSleep queda casi al final con menos del 1%, aunque es la variable con mayor relacion individual con el objetivo. La razon es que GastoTotal ya captura esa informacion, porque estar en criosuenio implica gasto cero. Al usarse en la raiz, para cuando el arbol podria preguntar por CryoSleep esa pregunta ya no aporta. No es inutil, es redundante dado el orden en que el arbol pregunta. Lo mismo pasa con ViajaSolo y EsNino, que quedaron sin usar por ser versiones simplificadas de TamGrupo y Age.

9. El programa hace predicciones

Hay dos formas, las dos desde la terminal. El modo interactivo (python main.py --interactivo) pide los datos de un pasajero y devuelve la prediccion con su probabilidad; los campos vacios se completan con las reglas de imputacion del entrenamiento y cada dato se valida, asi que no admite una edad negativa ni un grupo de cero personas. El modo por lotes (python main.py --kaggle) clasifica los 4277 pasajeros de test.csv, que son datos nuevos que no participaron ni en el entrenamiento ni en la evaluacion.

Sobre esas predicciones no se pueden calcular metricas porque no existen las etiquetas verdaderas. Lo

que si se verifica es que la proporcion predicha se parezca a la del entrenamiento, y asi es: alrededor del 50%. Si el modelo estuviera sesgado, esa proporcion se iria a un extremo.

10. Analisis y conclusiones

El arbol clasifica correctamente 77.48% de los pasajeros de prueba contra 50.36% del clasificador trivial. Esa diferencia de 27.12 puntos, junto con las 72 pruebas de correctitud, permite afirmar que el algoritmo aprende y que la implementacion es correcta. El desempenio esta equilibrado entre las dos clases y la brecha entrenamiento-prueba es de 3.03 puntos, asi que generaliza en lugar de memorizar.

Sobre las limitaciones: un arbol solo hace cortes perpendiculares a los ejes, una variable a la vez, asi que una frontera diagonal tendria que aproximarla con muchos escalones, y eso acota el techo alcanzable. Ademias la construccion es voraz, se toma la mejor division inmediata sin ver como afecta a los niveles siguientes, asi que no hay garantia de llegar al mejor arbol posible; encontrar el optimo es computacionalmente muy costoso y todas las implementaciones, incluidas las de las bibliotecas conocidas, usan esta misma estrategia. Tampoco es del todo estable: cambiar la semilla mueve la exactitud algunas decimas.

La extension natural seria entrenar muchos arboles sobre muestras distintas y promediar sus votos, que es la idea del random forest: reduce la inestabilidad y suele ganar algunos puntos, a cambio de perder la posibilidad de leer las reglas.

Programar el algoritmo desde cero deja claro que un arbol no es una caja negra: es una sola formula, la del Gini, aplicada de forma recursiva. Lo que mas trabajo lleva no es entrenar sino lo que lo rodea, o sea preparar los datos, evitar la fuga de informacion, elegir los hiperparametros sin contaminar la prueba y comprobar que el codigo hace lo que se supone. Los frenos al crecimiento resultaron lo mas determinante: dejando crecer el arbol hasta 20 niveles, el entrenamiento llega a 80.56% pero la prueba se queda en 77.41%, por debajo del 78.10% que se alcanza con 6. Todo ese crecimiento extra sirvio solo para ajustarse a datos ya conocidos, que es la definicion de no haber aprendido nada util.